



Automated Test Case Generation for Software Testing Using Generative AI

Ajay Bandi^(✉) , Harsha Sai Teja Nukala, Bhavya Tatavarthi,
and Amulya Boggavarapu

School of Computer Science and Information Systems, Northwest Missouri State
University, 800 University Dr, Maryville, MO 64468, USA
{AJAY,s564912,s566579,s567145}@nwmissouri.edu

Abstract. Software testing is a critical phase in the development life-cycle, ensuring the reliability and correctness of software systems. Traditional test case generation can be time-consuming and labor-intensive, often requiring significant manual effort. With the rapid advancement of generative AI, tools like ChatGPT and Gemini offer new possibilities for automating this process. This paper investigates the application of these AI-driven tools for test case generation, evaluating their effectiveness in achieving comprehensive code coverage across diverse programming problems. This paper investigates the application of generative AI tools, specifically ChatGPT and Gemini, for automating test case generation in software testing. By analyzing source code from 30 programming problems across various topics, including loops, conditionals, arrays, strings, and recursion, the study evaluates the tools' effectiveness in achieving comprehensive code coverage. The research addresses key questions regarding the quality, coverage, and efficiency of AI-generated test cases compared to manual efforts. The findings reveal that ChatGPT consistently outperforms Gemini in accuracy, adaptability, and handling of complex constructs, such as recursive algorithms and nested structures. While both tools reduce the manual effort required for test case generation, Gemini shows limitations in achieving full coverage for advanced scenarios. These results underscore the potential of generative AI to streamline software testing workflows, freeing developers to focus on higher-order problem-solving. However, the study also highlights the need for further refinement of these tools to enhance their reliability and robustness. This work provides a foundational step toward leveraging generative AI to transform software development and testing practices.

Keywords: Automated Test Case Generation · Software Testing with AI · Generative AI for Testing · AI-Powered Test Automation · Machine Learning in Software Testing · AI-Driven Test Case Generation

1 Introduction

Software testing is an essential phase in the software development lifecycle, ensuring that applications function as intended and meet quality standards. Tradi-

tional testing methodologies involve both manual and automated approaches to validate software correctness. While automated testing has significantly improved efficiency, manually crafting test cases remains a bottleneck, requiring domain expertise and extensive effort to ensure thorough coverage.

Recent advancements in artificial intelligence (AI) [3, 4] have introduced novel techniques for automating software testing. AI-powered tools leverage natural language processing (NLP) and machine learning to analyze code, predict potential failures, and generate test cases with minimal human intervention. AI-driven test case generation has the potential to enhance software reliability by automating tedious tasks, reducing human error, and improving overall test coverage [1, 7].

In the ever-evolving field of software development, ensuring the quality and reliability of code is paramount. Automated testing has become a cornerstone of this process, allowing developers to identify bugs and validate functionality efficiently. However, generating test cases manually remains a time-consuming and resource-intensive task, particularly when addressing complex problems and edge cases. The advent of artificial intelligence (AI) has introduced new opportunities to enhance this aspect of software testing, offering potential solutions that combine speed, accuracy, and adaptability.

This research explores the use of advanced AI tools, specifically ChatGPT and Gemini (Google Bard), for automating test case generation. These tools leverage natural language processing and machine learning techniques to analyze code and generate relevant test cases. The study seeks to evaluate their ability to achieve comprehensive code coverage, identify potential issues, and streamline the testing process. By focusing on diverse programming problems, this research aims to understand the practicality and effectiveness of integrating AI into traditional software testing workflows.

To conduct this study, a variety of problems were sourced from HackerRank, covering key programming topics such as conditional statements, loops, arrays, strings, and recursion. For each problem, the source code was written manually and its accuracy was validated using predefined test cases in HackerRank. The solutions for the same problems were then input into ChatGPT and Gemini, prompting these AI tools to generate the corresponding test cases. The generated test cases were tested in the NetBeans IDE to verify their accuracy, functionality, and ability to achieve full code coverage.

A detailed comparative analysis was performed to evaluate the performance of the AI tools. This included evaluating the quality of test cases, coverage metrics, handling of edge cases, and efficiency in generating results compared to manual efforts. The findings provide valuable insights into the strengths and limitations of using AI for automated testing, paving the way for further research in this domain.

By addressing the challenges associated with manual test case generation and exploring the potential of AI-based solutions, this study aims to contribute to the ongoing evolution of software development practices, offering a glimpse into the future of intelligent automated testing systems.

The remainder of this paper is organized as follows. Section 2 presents related work, discussing previous studies on AI-based test case generation and automated software testing. Section 3 outlines the research questions that guide this study. Section 4 describes the methodology, including the selection of programming problems, evaluation criteria, and experimental setup. Section 5 presents the results and analysis, highlighting the effectiveness of ChatGPT and Gemini in generating test cases. Section 6 provides a discussion of the findings, examining key trends, challenges, and implications of AI-driven test case generation. Finally, Sect. 7 concludes the study and outlines potential directions for future research.

2 Related Work

Recent studies have explored the transformative impact of generative AI (GenAI) on various aspects of software engineering, each addressing unique challenges and opportunities. Wu et al. [10] highlighted the dual utility of GenAI in software development, emphasizing its application in automating repetitive tasks such as boilerplate code generation and debugging, as well as its role in tackling complex scenarios like working with legacy code. By streamlining routine tasks, GenAI enables developers to focus on high-level problem-solving and innovation, while also assisting in creating unit tests and refining algorithms.

In contrast, Aleti et al. [2] examined the challenges of applying traditional software testing methods to GenAI systems, which often produce diverse and unpredictable outputs. The study proposed innovative solutions, including Test Suite Instance Space Adequacy (TISA) metrics and active learning techniques, to address issues like test adequacy and bias detection. This work underscores the need for new testing paradigms tailored to GenAI's unique characteristics.

Mock et al. [8] delved into integrating GenAI in Test-Driven Development (TDD), focusing on how tools like ChatGPT can automate production code generation and iterative testing based on pre-written test cases. The findings demonstrated that while GenAI can enhance productivity and test coverage, human oversight remains crucial to ensure the accuracy and reliability of outputs. This integration streamlines TDD workflows, enabling developers to concentrate on refining outputs and handling complex edge cases.

Rahman et al. [9] introduced GeneUS, a tool powered by GPT-4.0, to automate user story and test case generation in Agile software development. By leveraging a novel prompting approach, GeneUS improves clarity and reduces errors like hallucinations, significantly boosting productivity in user story creation and integration with tools such as Jira. The study highlighted the potential of GenAI to automate requirements analysis and improve workflow efficiency.

Finally, Coutinho et al. [6] conducted a pilot study to evaluate the role of GenAI in enhancing software development productivity. Participants reported benefits such as time optimization, streamlined workflows, and improved collaboration, alongside challenges like prompt precision and reliability. The study emphasized the capacity of GenAI to consolidate multiple tasks, from artifact

generation to formal documentation, into an efficient workflow, fostering creativity and collaboration.

Collectively, these studies demonstrate the multifaceted contributions of GenAI to software engineering. While Wu et al. [10] and Rahman et al. [9] focus on task automation and productivity enhancement, Aleti et al. [2] and Mock et al. [8] address the complexities of testing and TDD integration. Coutinho et al. [6] further expand on GenAI's collaborative potential. These works collectively highlight both the opportunities and challenges of adopting GenAI, underscoring the need for continuous innovation in tools and methodologies.

3 Research Questions

We considered problems across various programming constructs, including If-Else, For Loop, While Loop, Arrays, Strings, and Recursion. Under If-Else, the problems included Determine strangeness based on integer, Determine blood donation eligibility, Determine sports team eligibility based on age and height, Determine driving license eligibility based on age and vision score, and Determine university admission eligibility based on GPA and Entrance Exam Score. The For Loop category covered Multiplication Table - Easy, Generating Series - Easy, Find Digits - Intermediate, Circular Array, and Printing Patterns. For While Loop, the problems included Calculate Sum of Array Elements Using a While Loop, Find the Greatest Common Divisor (GCD) of Two Numbers Using a While Loop, Count Digits in a Number Using a While Loop, Reverse a Number Using a While Loop, and Calculate the Factorial of a Number Using a While Loop. The Arrays category comprised Calculate Sum of Array Elements, Find Maximum Element in an Array, Java Program to Reverse an Array, Count Even Numbers in an Array, and Count Positive and Negative Numbers in an Array. Under Strings, the problems involved Count Uppercase and Lowercase Letters in a String, Check if a String is a Palindrome, Count Words in a Sentence, Count Digits and Special Characters in a String, and Find the First Non-Repeating Character in a String. Lastly, for Recursion, we considered Fibonacci Series, Factorial, Staircase Climb, Sum of Digits, and Merge Sort.

RQ 1: How can large language models be utilized to create efficient and comprehensive test cases? Large Language Models (LLMs) like ChatGPT and Gemini can generate effective test cases by analyzing problem descriptions and source code to identify key inputs, outputs, and potential edge cases. They automate the creation of diverse test scenarios, including basic, boundary, and edge cases, ensuring comprehensive coverage. LLMs leverage natural language understanding to align test cases with functional goals and can refine outputs interactively based on developer feedback. Their scalability enables rapid generation of test cases for complex programs, significantly reducing manual effort. By tailoring test cases to specific domains, LLMs enhance the efficiency and precision of software testing workflows.

RQ 2: How do ChatGPT and Gemini compare in terms of test case generation quality, code coverage, and efficiency when used for automated software

testing? The comparison between ChatGPT and Gemini reveals distinct differences in their ability to generate test cases for coding problems. ChatGPT consistently provides high accuracy, achieving 100% code coverage across a variety of topics such as arrays, loops, and strings. Gemini, on the other hand, exhibits variability in performance, with test case accuracy ranging from 60% to 100%. While Gemini performs well in simpler problems, it faces challenges in handling more complex scenarios and edge cases, particularly in if-else and string-related tasks. This comparison demonstrates ChatGPT’s superior reliability for comprehensive test case generation, while Gemini shows potential for improvement in handling complex problem types.

4 Research Methodology

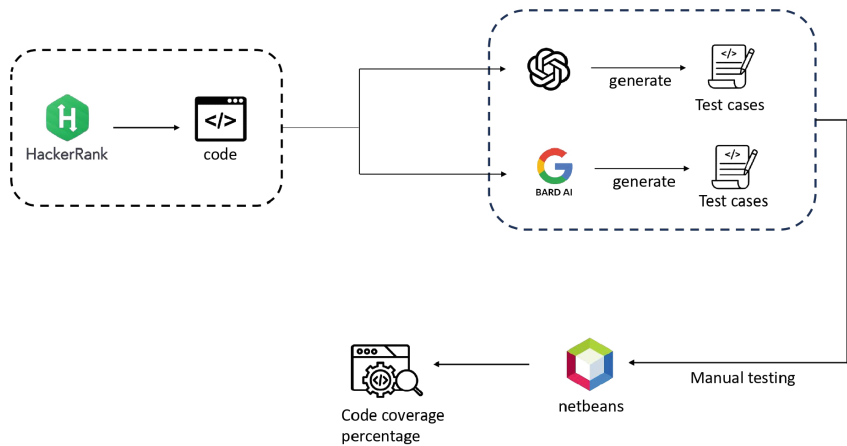


Fig. 1. Research Methodology

The research methodology is shown in Fig. 1. The different phases are detailed in this section. ChatGPT said: The problems selected for this study cover a diverse range of programming concepts, including conditional statements, loops, arrays, strings, and recursion. The selection criteria aimed to ensure a balanced mix of complexity, real-world applicability, and variation in logical structures. The chosen problems range from basic eligibility checks and arithmetic operations to more complex recursive and sorting algorithms. This diverse set allows for a comprehensive evaluation of generative AI tools in generating effective test cases across different programming paradigms.

4.1 Phase1-Problem Selection and Manual Code Development

In this phase, coding problems were selected from HackerRank across diverse topics such as conditional statements, loops, arrays, strings, and recursion. Each

problem was manually solved, and the solutions were tested using HackerRank’s predefined test cases to ensure accuracy and completeness. These manually written codes served as a reliable benchmark for evaluating AI-generated test cases.

4.2 Phase2-AI Test Case Generation

The manually written source code was then shared with ChatGPT and Google Bard (Gemini), two AI tools tasked with generating test cases. The AI-generated test cases included basic scenarios for standard inputs and edge cases to handle extreme or unusual conditions. This phase aimed to evaluate the AI’s understanding of problem statements and its capability to generate diverse and meaningful test scenarios.

4.3 Phase3-Test Case Execution and Analysis

The test cases generated by the AI tools were executed using NetBeans IDE on the manually written source code. This process assessed the test cases for code coverage, ensuring all functional areas like branches and loops were tested, and for correctness, verifying that potential bugs were accurately detected. A comparative analysis followed to evaluate the coverage, quality, and efficiency of test cases from both AI tools, shedding light on their effectiveness in automated testing workflows.

5 Results and Analysis

We followed the prompt structure [5] to generate test cases: “Here is the source code of the problem. Analyze it as a software tester and generate test cases to achieve 100% code coverage.” Table 1 presents the results for various programming concepts, including if-else statements, for and while loops, arrays, strings, and recursion. The source code was written in Java for a total of 30 HackerRank problems, with five problems on each topic. The questions are also included in the results table.

The integration of if-else statements and for loops forms a cornerstone of logical and repetitive operations in programming. If-else statements enable branching logic, facilitating decision-making based on conditions like input validation or error handling. For loops automate repetitive tasks, such as iterating through data sets or performing bulk calculations. By combining these constructs, the AI models were able to generate comprehensive test cases that included checks for boundary conditions and scenarios requiring iterative logic. For example, test cases were created to validate user inputs against predefined criteria and to iterate through complex data arrays effectively.

Table 1. Test Case Generation Results for Various Topics

Topic	No. of Questions	Name of the Question	HackerRank	ChatGPT	GeminiAI (Google Bard)
IF-ELSE	5	1. Determine Weirdness Based on an Integer	100%	100%	90%
		2. Determine Blood Donation Eligibility	100%	80%	100%
		3. Determine Sports Team Eligibility Based on Age and Height	100%	100%	70%
		4. Determine Driving License Eligibility Based on Age and Vision Score	100%	100%	60%
		5. Determine University Admission Eligibility Based on GPA and Entrance Exam Score	100%	100%	60%
FOR LOOP	5	1. Multiplication table-easy	100%	100%	100%
		2. Generating series-easy	100%	100%	100%
		3. Find digits-Intermediate	100%	100%	100%
		4. Circular array	100%	100%	100%
		5. Printing patterns	100%	75%	100%
WHILE LOOP	5	1. Calculate Sum of Array Elements Using a While Loop	100%	90%	80%
		2. Find the Greatest Common Divisor (GCD) of Two Numbers Using a While Loop	100%	100%	90%
		3. Count Digits in a Number Using a While Loop	100%	70%	80%
		4. Reverse a Number Using a While Loop	100%	100%	90%
		5. Calculate the Factorial of a Number Using a While Loop	100%	87.5%	80%
ARRAYS	5	1. Calculate Sum of Array Elements	100%	100%	90%
		2. Find Maximum Element in an Array	100%	100%	100%
		3. Java Program to Reverse an Array	100%	80%	100%
		4. Count Even Numbers in an Array	100%	100%	100%
		5. Count Positive and Negative Numbers in an Array	100%	100%	100%
STRINGS	5	1. Count Uppercase and Lowercase Letters in a String	100%	100%	60%
		2. Check if a String is a Palindrome	100%	100%	70%
		3. Count Words in a Sentence	100%	85%	40%
		4. Count Digits and Special Characters in a String	100%	100%	70%
		5. Find the First Non-Repeating Character in a String	100%	90%	60%
RECURSION	5	1. Fibonacci series	100%	100%	100%
		2. Factorial	100%	100%	100%
		3. Staircase climb	100%	80%	100%
		4. Sum of digits	100%	100%	100%
		5. Merge Sort	100%	100%	60%

While loops provide dynamic iteration capabilities, enabling processes to continue until a specified condition is met. This flexibility is crucial for tasks like searching unsorted datasets or responding to unpredictable input conditions. Arrays, as foundational data structures, allow for the storage and manipula-

tion of ordered collections. The generated test cases addressed various scenarios, including handling empty arrays, single-element arrays, and boundary indices. The AI tools demonstrated their capability to identify potential edge cases by leveraging the dynamic nature of while loops in conjunction with the structured organization provided by arrays.

Strings serve as versatile data types in programming, frequently employed in applications involving text processing, data validation, or pattern recognition. Test cases were designed to validate string operations, such as handling empty strings, processing special characters, and managing varying string lengths. Recursion, a more complex concept, involves functions calling themselves to solve problems iteratively. This approach is especially useful for problems like factorial computation or traversing hierarchical structures. The AI-generated test cases effectively examined recursion by addressing challenges such as infinite recursion prevention and base case validation, demonstrating their strength in tackling computationally intensive problems.

The selected problems ranged from basic constructs, such as loops and arrays, to advanced topics, including recursion and dynamic memory management. While simpler problems were handled adeptly by both models, the complexity of recursive algorithms highlighted deficiencies, particularly for Gemini, in achieving comprehensive coverage and addressing edge cases effectively.

ChatGPT consistently outperformed Gemini in terms of accuracy, efficiency, and comprehensiveness of test cases. Gemini displayed strength in simpler scenarios but struggled with advanced constructs, particularly recursion and complex conditionals. This performance gap underscores ChatGPT's robustness and adaptability in diverse scenarios. Gemini's tendency to generate redundant test cases for simpler problems. Both models faced challenges in generating test cases for nested structures and recursive functions, with ChatGPT demonstrating relatively more consistent performance.

Large language models such as ChatGPT and Gemini demonstrate significant potential in generating test cases for various programming constructs. ChatGPT excels in handling diverse constructs, including loops, conditionals, and advanced topics like recursion. Its ability to provide comprehensive code coverage, even for complex problems, showcases its adaptability and strength. Gemini, while competent with simpler constructs like loops and arrays, struggles with advanced scenarios, particularly recursive algorithms and nested structures. Overall, ChatGPT's consistency and adaptability make it a more effective tool for generating test cases across diverse programming challenges.

The strengths of ChatGPT lie in its prompt adherence, ability to prioritize edge cases, and consistent performance across varying complexity levels. Its adaptability ensures robust test case generation for intricate problems involving recursion and dynamic conditions. Gemini, on the other hand, performs well in generating test cases for simpler problems, demonstrating efficiency and speed. However, its weaknesses become evident in handling complex constructs, where it often fails to achieve comprehensive coverage or address edge cases effectively.

These limitations highlight the need for further refinement of Gemini's algorithms to improve its reliability for advanced test case generation scenarios.

6 Discussion

The results showed that both ChatGPT and Gemini could generate test cases that worked well for basic programming problems. They handled simple logic, such as loops and conditional statements, quite well. However, when the problems became more complex, the test cases were not always complete. In many cases, the AI tools missed important edge cases, which are critical for ensuring that a program works correctly in all situations. This means that while AI can help speed up test case generation, human review is still necessary to make sure all possible scenarios are covered.

Another key finding was that AI-generated test cases often achieved good basic coverage but struggled with more detailed testing, such as covering all possible decision paths in a program. Sometimes, test cases included unnecessary repetitions, making them less efficient than manually written ones. Additionally, when the same prompt was given multiple times, the AI models sometimes generated different test cases each time. This inconsistency can be a challenge when trying to create reliable and repeatable tests.

Overall, AI tools like ChatGPT and Gemini can be useful for generating test cases quickly, but they are not perfect. Developers should use them as a starting point rather than relying on them completely. Future improvements in AI may help overcome some of these issues, but for now, human expertise is still essential in software testing.

7 Conclusions and Future Work

In this study, we explored how generative AI tools like ChatGPT and Gemini can assist in automating test case generation for software testing. By applying these tools to a variety of programming problems, we found that they can significantly reduce the manual effort involved in creating test cases while ensuring comprehensive code coverage. ChatGPT stood out for its consistent performance, especially in handling complex problems like recursion and nested structures, whereas Gemini showed potential but struggled with more advanced scenarios.

Our findings highlight the value of generative AI in streamlining software testing processes, making them faster and more efficient. At the same time, we identified areas where these tools need improvement, particularly in addressing edge cases and complex constructs.

Generative AI has the potential to transform software development by freeing developers to focus on creative problem-solving and innovation. Future work should aim to refine these tools further, enabling them to handle more complex challenges and expanding their application to other areas of software engineering.

Future research should focus on enhancing the accuracy and reliability of generative AI tools in test case generation, particularly in handling edge cases

and intricate program logic. Further studies could explore integrating these AI models with existing testing frameworks to improve automation and adaptability. Additionally, expanding the evaluation to real-world software projects and diverse programming paradigms will provide deeper insights into their practical applicability. Addressing these challenges will be crucial for fully leveraging generative AI in software testing and broader software engineering workflows.

References

1. Alenezi, M., Akour, M.: Ai-driven innovations in software engineering: a review of current practices and future directions. *Appl. Sci.* **15**(3), 1344 (2025)
2. Aleti, A.: Software testing of generative AI systems: challenges and opportunities. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), pp. 4–14. IEEE (2023)
3. Bandi, A., Adapa, P., Kuchi, Y.: The power of generative AI: a review of requirements, models, input-output formats, evaluation metrics, and challenges. *Future Internet* **15**(8), 260 (2023)
4. Bandi, A., Kagitha, H.: A case study on the generative AI project life cycle using large language models. In: Proceedings of 39th International Conference, vol. 98, pp. 189–199 (2024)
5. Bandi, A., Zeng, R.: Evaluation of the effectiveness of prompts and generative ai responses. In: International Conference on Computer Applications in Industry and Engineering, pp. 56–69. Springer, Heidelberg (2024). https://doi.org/10.1007/978-3-031-76273-4_5
6. Coutinho, M., Marques, L., Santos, A., Dahia, M., França, C., de Souza Santos, R.: The role of generative AI in software development productivity: a pilot case study. In: Proceedings of the 1st ACM International Conference on AI-Powered Software, pp. 131–138 (2024)
7. Hourani, H., Hammad, A., Lafi, M.: The impact of artificial intelligence on software testing. In: 2019 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT), pp. 565–570. IEEE (2019)
8. Mock, M., Melegati, J., Russo, B.: Generative AI for test driven development: preliminary results. In: International Conference on Agile Software Development, pp. 24–32. Springer, Heidelberg (2024). https://doi.org/10.1007/978-3-031-72781-8_3
9. Ramler, R., Putschögl, W., Winkler, D.: Automated testing of industrial automation software: practical receipts and lessons learned. In: Proceedings of the 1st International Workshop on Modern Software Engineering Methods for Industrial Automation, pp. 7–16 (2014)
10. Wu-Gehbauer, M., Rosenkranz, C.: Unlocking the potential of generative artificial intelligence: a case study in software development (2024)